

CPE 342 Machine Learning

Assignment 1: Training Models

OLS Regression

67070501042 วิศิษฐ์ สุวรรณเนา

ข้อมูลที่กำหนดให้เป็นตัวเลขงบประมาณโฆษณา (X , หน่วย: พันดอลลาร์) และยอดขาย (Y , หน่วย: พันหน่วย) จาก 10 เดือน ดังนี้

Month	Advertising Budget X (\$1,000s)	Sales Y (1,000 units)
1	3.0	6.0
2	5.0	9.0
3	2.0	4.0
4	7.0	10.0
5	8.0	12.0
6	1.0	3.0
7	4.0	7.0
8	6.0	8.0
9	9.0	13.0
10	10.0	15.0

Task 1: Implement OLS Regression

1a) Fit Simple Linear Regression ด้วยวิธี OLS

แบบจำลอง: สมมติว่าความสัมพันธ์ระหว่าง X กับ Y เป็นเชิงเส้น (linear) กำหนดให้ $\hat{Y}_i = \alpha + \beta X_i$ โดยที่ α คือ intercept และ β คือ slope

แทน $Y_i = \beta X_i + \alpha$ ลงในสมการ error $\varepsilon_i = Y_i - \hat{Y}_i$ จะได้

$$\varepsilon_i = Y_i - (\alpha + \beta X_i)$$

หลักการของ OLS: พยายามลดผลรวมของกำลังสองของ error (Least Squares Function) S :

$$S(\alpha, \beta) = \sum_{i=1}^n \varepsilon_i^2 = \sum_{i=1}^n (Y_i - \alpha - \beta X_i)^2$$

ฟังก์ชัน S จะมีค่าน้อยที่สุดเมื่อ $\frac{\partial S}{\partial \alpha} = 0$ และ $\frac{\partial S}{\partial \beta} = 0$

การหา partial derivatives:

$$\frac{\partial S}{\partial \alpha} = -2 \sum_{i=1}^n (Y_i - \alpha - \beta X_i) = 0 \quad (1)$$

$$\frac{\partial S}{\partial \beta} = -2 \sum_{i=1}^n (Y_i - \alpha - \beta X_i) X_i = 0 \quad (2)$$

จากสมการ (1): $\sum_{i=1}^n Y_i - n\alpha - \beta \sum_{i=1}^n X_i = 0 \implies$ **Normal Equation 1:**

$$n\alpha + \beta \sum_{i=1}^n X_i = \sum_{i=1}^n Y_i$$

จากสมการ (2): $\sum_{i=1}^n X_i Y_i - \alpha \sum_{i=1}^n X_i - \beta \sum_{i=1}^n X_i^2 = 0 \implies$ **Normal Equation 2:**

$$\alpha \sum_{i=1}^n X_i + \beta \sum_{i=1}^n X_i^2 = \sum_{i=1}^n X_i Y_i$$

สามารถเขียน Normal Equation ในรูปแบบ Matrix ได้ดังนี้:

$$\begin{bmatrix} n & \sum X_i \\ \sum X_i & \sum X_i^2 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \sum Y_i \\ \sum X_i Y_i \end{bmatrix}$$

คำนวณค่า summation จากข้อมูล:

i	X_i	Y_i	$X_i Y_i$	X_i^2
1	3.0	6.0	18.0	9.0
2	5.0	9.0	45.0	25.0
3	2.0	4.0	8.0	4.0
4	7.0	10.0	70.0	49.0
5	8.0	12.0	96.0	64.0
6	1.0	3.0	3.0	1.0
7	4.0	7.0	28.0	16.0
8	6.0	8.0	48.0	36.0
9	9.0	13.0	117.0	81.0
10	10.0	15.0	150.0	100.0
$\sum$	55.0	87.0	583.0	385.0

ดังนั้น:

$$n = 10, \quad \sum X_i = 55, \quad \sum Y_i = 87, \quad \sum X_i Y_i = 583, \quad \sum X_i^2 = 385$$

สรุป 1a: จากข้อมูลข้างต้น จะได้ระบบสมการ:

$$\begin{bmatrix} 10 & 55 \\ 55 & 385 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} 87 \\ 583 \end{bmatrix}$$

1b) คำนวณ Regression Coefficients

เราสามารถหาค่า α (intercept) และ β (slope) ได้หลายวิธี:

วิธีที่ 0: สูตร S_{xx}, S_{xy} (Closed-form จาก Normal Equations)

จาก Normal Equations สามารถแก้หาค่า β ได้โดยตรงด้วยการนิยามทั้งสองแบบ:

$$S_{xx} = \sum_{i=1}^n (X_i - \bar{X})^2 = \sum X_i^2 - \frac{(\sum X_i)^2}{n} = 385 - \frac{55^2}{10} = 385 - 302.5 = 82.5$$

$$S_{xy} = \sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y}) = \sum X_i Y_i - \frac{\sum X_i \sum Y_i}{n} = 583 - \frac{(55)(87)}{10} = 583 - 478.5 = 104.5$$

จะได้:

$$\beta = \frac{S_{xy}}{S_{xx}} = \frac{104.5}{82.5} \approx 1.2667, \quad \alpha = \bar{Y} - \beta \bar{X} = 8.7 - (1.2667)(5.5) \approx 1.7333$$

วิธีนี้ให้ผลลัพธ์ที่ตรงกันกับทุกวิธีอื่น แต่คำนวณได้เร็วกว่าเพราะไม่ต้องแก้ระบบสมการ

วิธีที่ 1: Cramer's Rule

จากระบบสมการ:

$$10\alpha + 55\beta = 87$$

$$55\alpha + 385\beta = 583$$

หาค่า Determinant:

$$\Delta = \det \begin{bmatrix} 10 & 55 \\ 55 & 385 \end{bmatrix} = (10)(385) - (55)(55) = 3850 - 3025 = 825$$

$$\Delta_\alpha = \det \begin{bmatrix} 87 & 55 \\ 583 & 385 \end{bmatrix} = (87)(385) - (55)(583) = 33495 - 32065 = 1430$$

$$\Delta_\beta = \det \begin{bmatrix} 10 & 87 \\ 55 & 583 \end{bmatrix} = (10)(583) - (87)(55) = 5830 - 4785 = 1045$$

จะได้:

$$\alpha = \frac{\Delta_\alpha}{\Delta} = \frac{1430}{825} = \frac{26}{15} \approx 1.7333$$

$$\beta = \frac{\Delta_\beta}{\Delta} = \frac{1045}{825} = \frac{19}{15} \approx 1.2667$$

วิธีที่ 2: Pseudoinverse Method

กำหนด $A = \begin{bmatrix} 1 & X_1 \\ 1 & X_2 \\ \vdots & \vdots \\ 1 & X_n \end{bmatrix}$, $Y = \begin{bmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{bmatrix}$, และ $c = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$

จากสมการ $Ac = Y$ เราใช้ Left Pseudoinverse:

$$c = (A^T A)^{-1} A^T Y$$

จากตารางคำนวณ:

$$A^T A = \begin{bmatrix} n & \sum X_i \\ \sum X_i & \sum X_i^2 \end{bmatrix} = \begin{bmatrix} 10 & 55 \\ 55 & 385 \end{bmatrix}$$

$$A^T Y = \begin{bmatrix} \sum Y_i \\ \sum X_i Y_i \end{bmatrix} = \begin{bmatrix} 87 \\ 583 \end{bmatrix}$$

หา $(A^T A)^{-1}$:

$$(A^T A)^{-1} = \frac{1}{825} \begin{bmatrix} 385 & -55 \\ -55 & 10 \end{bmatrix}$$

คำนวณหา c :

$$\begin{aligned} c &= \frac{1}{825} \begin{bmatrix} 385 & -55 \\ -55 & 10 \end{bmatrix} \begin{bmatrix} 87 \\ 583 \end{bmatrix} \\ &= \frac{1}{825} \begin{bmatrix} (385)(87) + (-55)(583) \\ (-55)(87) + (10)(583) \end{bmatrix} \\ &= \frac{1}{825} \begin{bmatrix} 33495 - 32065 \\ -4785 + 5830 \end{bmatrix} \\ &= \frac{1}{825} \begin{bmatrix} 1430 \\ 1045 \end{bmatrix} = \begin{bmatrix} 1430/825 \\ 1045/825 \end{bmatrix} = \begin{bmatrix} 26/15 \\ 19/15 \end{bmatrix} \end{aligned}$$

สรุป 1b: ได้ค่าสัมประสิทธิ์ดังนี้:

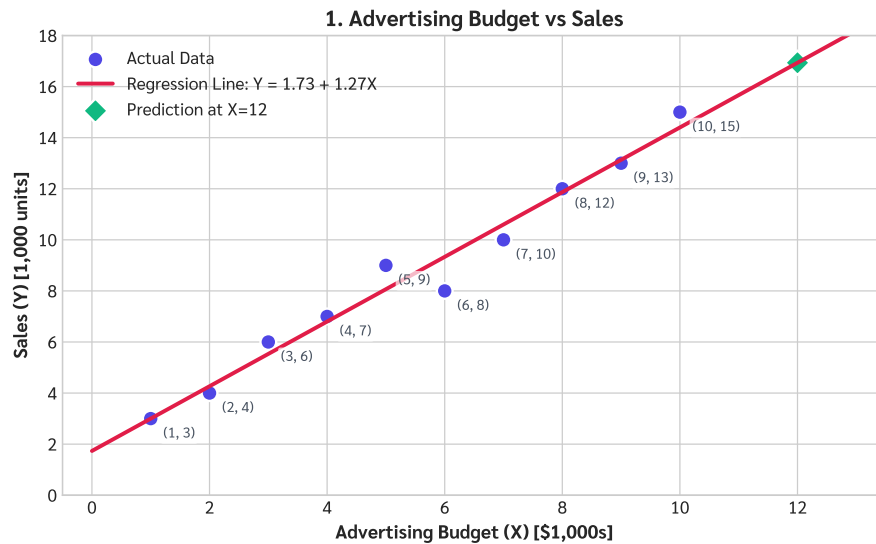
- **Intercept (α)** = $\frac{26}{15} \approx 1.7333$
- **Slope (β)** = $\frac{19}{15} \approx 1.2667$

1c) สมการ Fitted Line

นำค่า α และ β ที่คำนวณได้ไปแทนในสมการ $\hat{Y} = \alpha + \beta X$

สรุป 1c: สมการเส้นตรงพยากรณ์คือ

$$\hat{Y} = 1.7333 + 1.2667X$$



รูปที่ 1: OLS Linear Regression Analysis (Advertising Budget vs Sales)
แสดงข้อมูลจริงและเส้นพยากรณ์ $\hat{Y} = 1.7333 + 1.2667X$

Task 2: Interpret the Results

2a) ความหมายของ Slope และ Intercept

ความหมายของ Slope ($\beta \approx 1.2667$):

Slope บ่งบอกว่ายอดขาย (Sales) ถูกคาดการณ์ว่าจะเพิ่มตามค่าโฆษณา (Advertising) ที่เพิ่มขึ้น นั่นคือ ทุก ๆ \$1,000 ของ advertising ที่เพิ่มขึ้น จะทำให้ยอดขาย (Sales) เพิ่มขึ้นประมาณ **1,267 หน่วย** ค่านี้เป็นความสัมพันธ์เชิงสถิติ (association) ไม่ใช่การพิสูจน์ว่าโฆษณาเป็นสาเหตุ (causation) ของยอดขายที่เพิ่มขึ้น

ความหมายของ Intercept ($\alpha \approx 1.7333$):

จุดตัดแกน Y บ่งบอกยอดขายที่คาดการณ์ไว้เมื่อไม่มีต้นทุนค่าโฆษณา กล่าวคือ ถ้าบริษัทไม่ทำโฆษณาเลย ($X = 0$) จะสามารถคาดการณ์ยอดขายได้ประมาณ **1,733 หน่วย**

2b) Predict ยอดขายเมื่องบโฆษณา = \$12,000

แทน $X = 12$ ลงในสมการที่ได้:

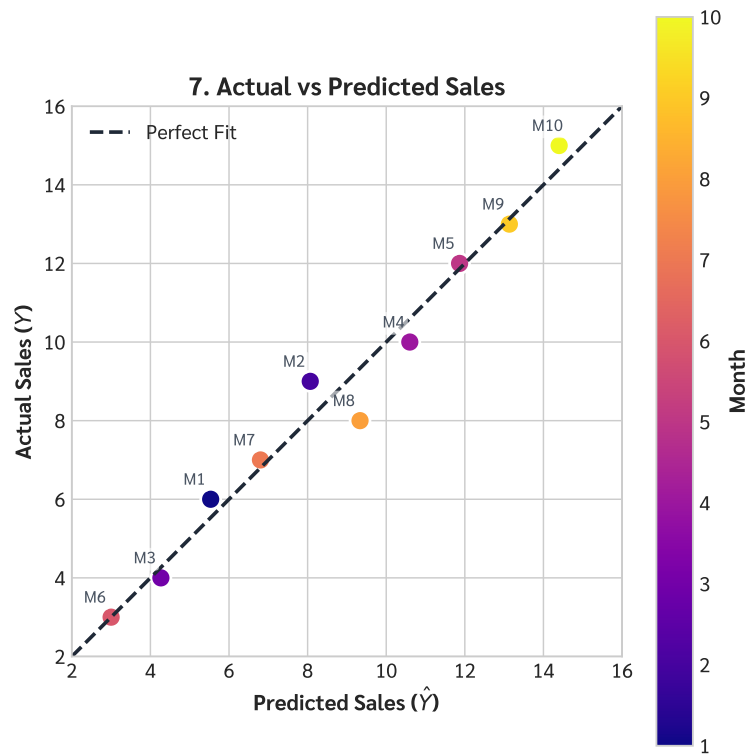
$$\hat{Y} = 1.7333 + 1.2667(12)$$

$$\hat{Y} = 1.7333 + 15.2004$$

$$\hat{Y} \approx 16.9333$$

สรุป 2b: ถ้าต้นทุนในการโฆษณาคือ \$12,000 คาดการณ์ยอดขายได้ประมาณ **16.93 พันหน่วย** (ประมาณ 16,933 หน่วย)

ข้อควรระวัง: $X = 12$ อยู่นอกช่วงข้อมูลที่ใช้ fit โมเดล (ค่าสูงสุดใน dataset คือ $X = 10$) การทำนายนี้จึงเป็น **extrapolation** ซึ่งโมเดลไม่สามารถยืนยันได้ว่าความสัมพันธ์เชิงเส้นยังคงดำเนินต่อไปที่ระดับนี้



รูปที่ 2: Actual vs Predicted Sales
 แสดงความแม่นยำของการพยากรณ์โดยเปรียบเทียบค่าพยากรณ์กับค่าจริง

Task 3: Model Evaluation

3a) คำนวณ R-squared (R^2)

R^2 วัดความเหมาะสมของแบบจำลอง (Goodness of fit) นิยามเป็น:

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

โดย:

- $SS_{\text{res}} = \sum (Y_i - \hat{Y}_i)^2$
- $SS_{\text{tot}} = \sum (Y_i - \bar{Y})^2$

คำนวณ: (โดย $\bar{Y} = 8.7$)

i	X_i	Y_i	$\hat{Y}_i = 1.7333 + 1.2667X_i$	$e_i = Y_i - \hat{Y}_i$	e_i^2	$(Y_i - \bar{Y})^2$
1	3.0	6.0	5.5333	0.4667	0.2178	7.2900
2	5.0	9.0	8.0667	0.9333	0.8711	0.0900
3	2.0	4.0	4.2667	-0.2667	0.0711	22.0900
4	7.0	10.0	10.6000	-0.6000	0.3600	1.6900
5	8.0	12.0	11.8667	0.1333	0.0178	10.8900
6	1.0	3.0	3.0000	0.0000	0.0000	32.4900
7	4.0	7.0	6.8000	0.2000	0.0400	2.8900
8	6.0	8.0	9.3333	-1.3333	1.7778	0.4900
9	9.0	13.0	13.1333	-0.1333	0.0178	18.4900
10	10.0	15.0	14.4000	0.6000	0.3600	39.6900
Σ					3.7333	136.1000

จากตาราง: $SS_{\text{res}} = 3.7333$, $SS_{\text{tot}} = 136.1000$

แทนค่า:

$$R^2 = 1 - \frac{3.7333}{136.10} = 1 - 0.0274 = 0.9726$$

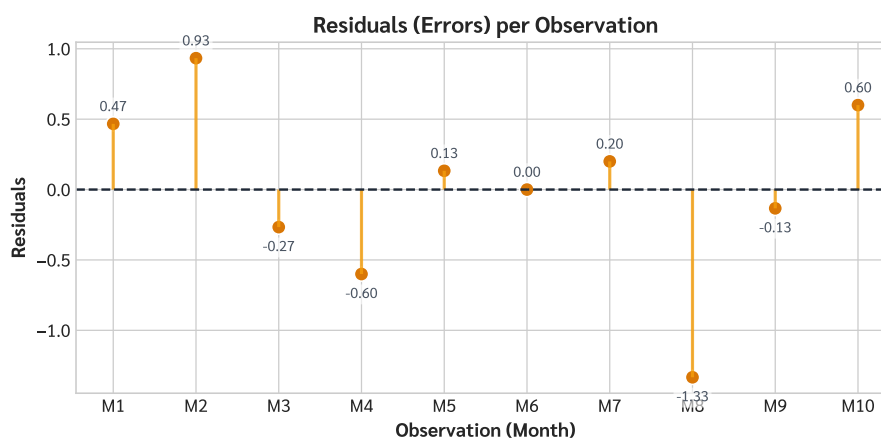
สรุป 3a: $R^2 \approx 0.9726$ หรือประมาณ 97.26%

3b) ดีความค่า R-squared

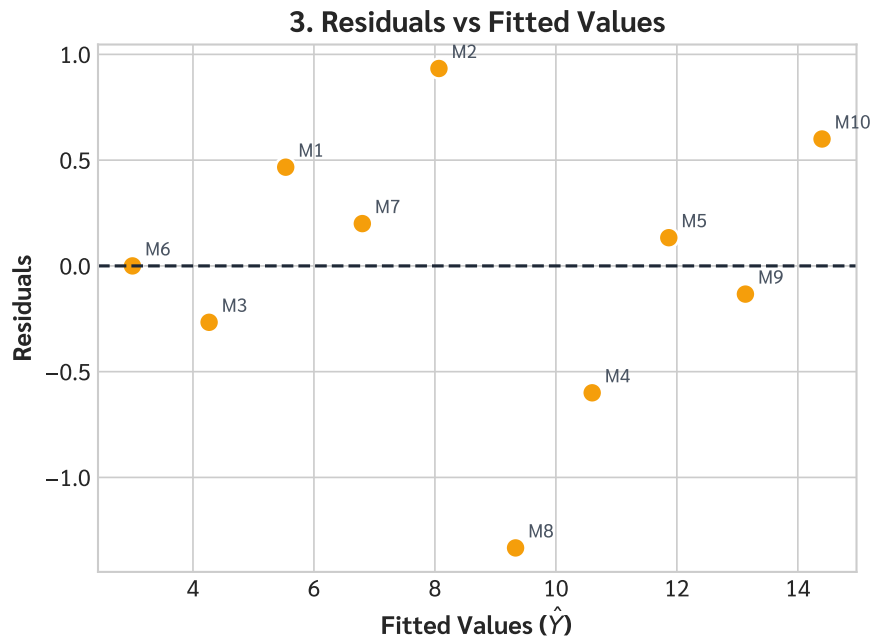
สรุป 3b: ค่า R-squared ที่ 0.9726 หมายความว่า ประมาณ **97.26%** ของการเปลี่ยนแปลงของยอดขาย (Sales) สามารถถูกอธิบายได้ด้วยงบประมาณโฆษณา (Advertising Budget) จากตาราง residuals ข้างต้น พบว่า $\bar{e} = 0$ (residual mean เท่ากับศูนย์พอดี แสดงว่าโมเดลไม่มี bias) โดย Standard Deviation ของ residuals อยู่ที่ $s_e = 0.6441$ พันหน่วย (เทียบกับค่าเฉลี่ยได้ดี เพราะ residual สูงสุดใน dataset คือ $|e_8| = 1.33$)

สิ่งที่ R^2 สูงไม่ได้บ่งชี้: ว่างบโฆษณาเป็นสาเหตุของยอดขาย หรือรับประกันว่าโมเดลจะพยากรณ์ได้ดีเสมอในอนาคต

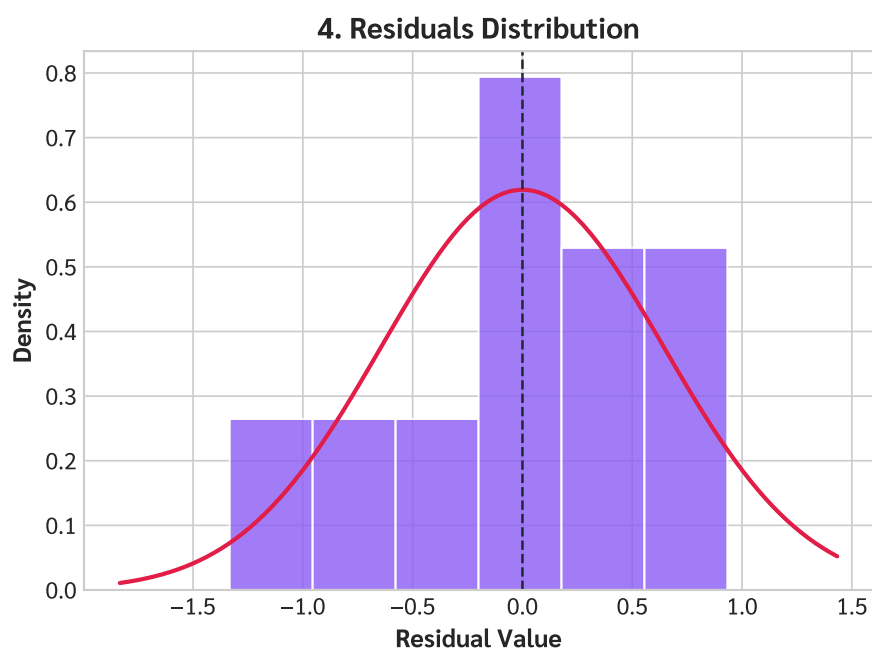
Diagnostic Plots (วิเคราะห์ความสัมพันธ์เชิงลึก)



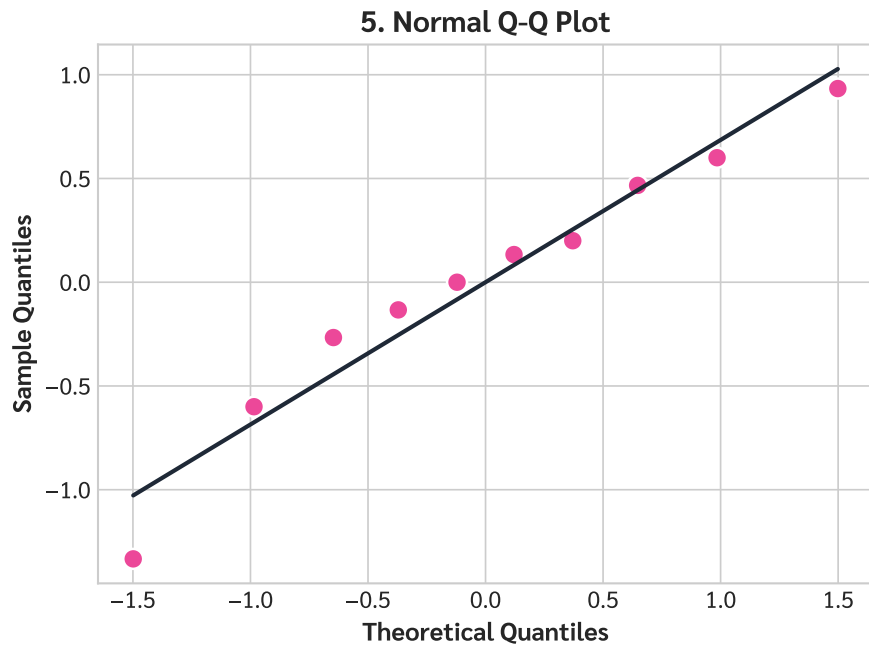
รูปที่ 3: **Residuals Lollipop Chart:** แสดงขนาดของความคลาดเคลื่อน (Residual) ของแต่ละจุดข้อมูล (M1-M10) ช่วยให้เห็นชัดเจนว่าจุดไหนที่โมเดลทำนายคลาดเคลื่อนมากที่สุด (เช่น M8 มีค่า Residual ต่ำกว่าศูนย์มากที่สุด)



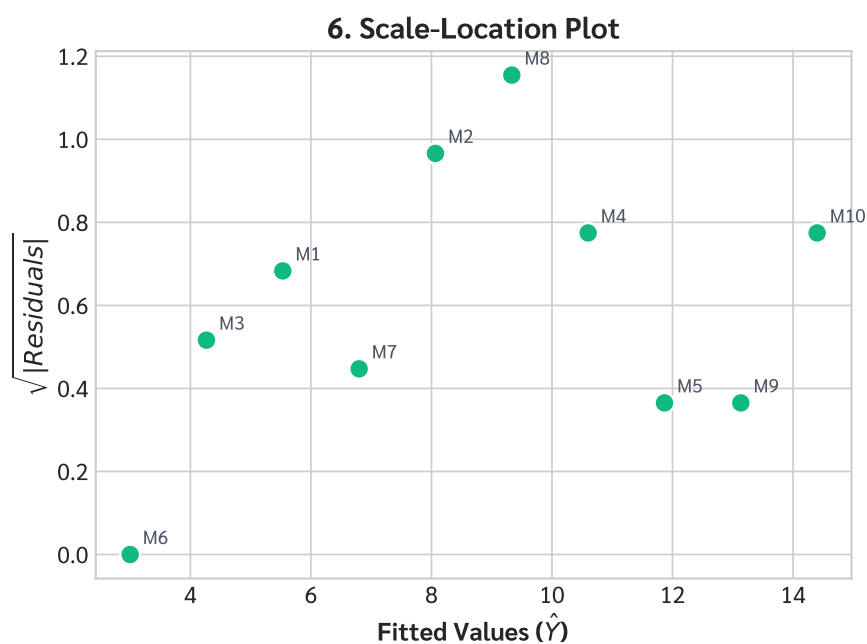
รูปที่ 4: **Residuals vs Fitted Values:** ใช้ตรวจสอบรูปแบบความคลาดเคลื่อน หากจุดกระจายตัวแบบสุ่มรอบๆ เส้น 0 แสดงว่าโมเดล Linear เหมาะสม ไม่มีปัญหา non-linearity



รูปที่ 5: **Residuals Distribution:**
 ตรวจสอบการกระจายตัวของความคลาดเคลื่อนว่าใกล้เคียงการแจกแจงแบบปกติ (Normal Distribution) หรือไม่ ซึ่งเป็นสมมติฐานสำคัญของ OLS



รูปที่ 6: **Normal Q-Q Plot:** ใช้เปรียบเทียบการกระจายตัวของ Residuals กับ Normal Distribution ในเชิง Quantile หากจุดส่วนใหญ่อยู่บนเส้นทแยงมุม แสดงว่า Residuals มีการแจกแจงแบบปกติ



รูปที่ 7: **Scale-Location Plot:** ใช้ตรวจสอบสมมติฐาน Homoscedasticity (ความแปรปรวนคงที่) หากจุดกระจายตัวสม่ำเสมอโดยไม่มีรูปแบบที่ชัดเจน แสดงว่าความแปรปรวนของความคลาดเคลื่อนคงที่ตลอดทุกช่วงค่าพยากรณ์

สรุปผลลัพธ์

ข้อ	คำตอบ
1a	ระบบสมการ $10\alpha + 55\beta = 87$ และ $55\alpha + 385\beta = 583$
1b	Intercept (α) ≈ 1.7333 , Slope (β) ≈ 1.2667
1c	สมการ Fitted Line: $\hat{Y} = 1.7333 + 1.2667X$
2a	Slope: เพิ่มงบประมาณ \$1,000 ทำให้ยอดขายเพิ่ม 1,267 หน่วย Intercept: หากไม่มีงบประมาณเลย ยอดขายคาดการณ์คือ 1,733 หน่วย
2b	เมื่องบประมาณ = \$12,000 คาดการณ์ยอดขายได้ประมาณ 16,933 หน่วย
3a	$R^2 \approx 0.9726$ (97.26%)
3b	97.26% ของยอดขายสามารถอธิบายได้ด้วยงบประมาณ (โมเดลมีความเหมาะสมดีเยี่ยม)

Appendix: Jupyter Notebook Output

ส่วนนี้คือโค้ด Python แบบเต็มและผลลัพธ์ (Output) ที่รันจริงจาก Assignment_1_OLS.ipynb เพื่อแสดงวิธีการคำนวณและสร้างกราฟทั้งหมด

Jupyter Notebook: Assignment_1_OLS.ipynb

โค้ดและผลลัพธ์ทั้งหมดด้านล่างเป็นการรันจริงจาก Jupyter Notebook

In [1]:

```
1 # Advertising budget (X, in $1,000s) and Sales (Y, in 1,000 units)
2 X = np.array([3., 5., 2., 7., 8., 1., 4., 6., 9., 10.])
3 Y = np.array([6., 9., 4., 10., 12., 3., 7., 8., 13., 15.])
4 n = len(X)
5 months = np.arange(1, n + 1)
6
7 print(f"n = {n}")
8 print(f"X = {X}")
9 print(f"Y = {Y}")
```

Out[1]:

```
n = 10
X = [ 3.  5.  2.  7.  8.  1.  4.  6.  9. 10.]
Y = [ 6.  9.  4. 10. 12.  3.  7.  8. 13. 15.]
```

In [2]:

```
1 sum_X = X.sum()
2 sum_Y = Y.sum()
3 sum_XY = (X * Y).sum()
4 sum_X2 = (X ** 2).sum()
5
6 print("Summation table:")
7 print(f"  sum(X)  = {sum_X}")
8 print(f"  sum(Y)  = {sum_Y}")
9 print(f"  sum(XY) = {sum_XY}")
10 print(f"  sum(X2) = {sum_X2}")
11 print()
12 print("Normal Equation (matrix form):")
13 print(f"  [{n:2d}  {sum_X:5.1f}] [alpha]  [{sum_Y:5.1f}]")
14 print(f"  [{sum_X:2.0f}  {sum_X2:5.1f}] [beta ] = [{sum_XY:5.1f}]" )
```

Out[2]:

Summation table:

```
sum(X)   = 55.0
sum(Y)   = 87.0
sum(XY)  = 583.0
sum(X2)  = 385.0
```

Normal Equation (matrix form):

```
[10  55.0] [alpha]  [ 87.0]
[55 385.0] [beta ] = [583.0]
```

In [3]:

```
1 X_bar = X.mean()    # 5.5
2 Y_bar = Y.mean()    # 8.7
3
4 Sxx = np.sum((X - X_bar) ** 2)          # 82.5
5 Sxy = np.sum((X - X_bar) * (Y - Y_bar)) # 104.5
6
7 beta  = Sxy / Sxx          # 1.266667
8 alpha = Y_bar - beta * X_bar      # 1.733333
9
10 print(f"X_bar = {X_bar}, Y_bar = {Y_bar}")
11 print(f"Sxx = {Sxx}, Sxy = {Sxy}")
12 print(f"beta (slope)      = {beta:.6f}")
13 print(f"alpha (intercept) = {alpha:.6f}")
14 print(f"Fitted line: Y_hat = {alpha:.4f} + {beta:.4f} * X")
15 assert np.isclose(beta, 19/15)
16 assert np.isclose(alpha, 26/15)
```

Out[3]:

```
X_bar = 5.5, Y_bar = 8.7
Sxx = 82.5, Sxy = 104.5
beta (slope)      = 1.266667
alpha (intercept) = 1.733333
Fitted line: Y_hat = 1.7333 + 1.2667 * X
```

In [4]:

```
1 A = np.array([[n, sum_X], [sum_X, sum_X2]], dtype=float)
2 b = np.array([sum_Y, sum_XY], dtype=float)
3
4 det_A      = np.linalg.det(A)
5 det_alpha  = np.linalg.det(np.column_stack([b, A[:, 1]]))
```

```

6 det_beta      = np.linalg.det(np.column_stack([A[:, 0], b]))
7
8 alpha_cr = det_alpha / det_A
9 beta_cr  = det_beta / det_A
10 print(f"Cramer's Rule → alpha = {alpha_cr:.6f}, beta = {beta_cr:.6f}")
11 assert np.isclose(alpha_cr, alpha) and np.isclose(beta_cr, beta)
12 print("[OK] Matches Sxx/Sxy result")

```

Out[4]:

```

Cramer's Rule → alpha = 1.733333, beta = 1.266667
[OK] Matches Sxx/Sxy result

```

In [5]:

```

1 A_mat = np.column_stack([np.ones(n), X]) # design matrix [1 | X]
2 coeffs = np.linalg.inv(A_mat.T @ A_mat) @ A_mat.T @ Y
3 alpha_ps, beta_ps = coeffs
4 print(f"Pseudoinverse → alpha = {alpha_ps:.6f}, beta = {beta_ps:.6f}")
5 assert np.isclose(alpha_ps, alpha) and np.isclose(beta_ps, beta)
6 print("[OK] Matches all previous results")

```

Out[5]:

```

Pseudoinverse → alpha = 1.733333, beta = 1.266667
[OK] Matches all previous results

```

In [6]:

```

1 Y_hat      = alpha + beta * X
2 residuals = Y - Y_hat
3
4 print(f"Fitted line: Y_hat = {alpha:.4f} + {beta:.4f} * X")
5 print()
6 print(f"{'Month':>5} {'X':>5} {'Y':>5} {'Y_hat':>8} {'e_i':>8}")
7 print("-" * 40)
8 for i in range(n):
9     print(f"    {i+1:2d}    {X[i]:4.1f}    {Y[i]:4.1f}    {Y_hat[i]:7.4f}
          {residuals[i]:+7.4f}")

```

Out[6]:

```

Fitted line: Y_hat = 1.7333 + 1.2667 * X

Month      X      Y      Y_hat      e_i

```

1	3.0	6.0	5.5333	+0.4667
2	5.0	9.0	8.0667	+0.9333
3	2.0	4.0	4.2667	-0.2667
4	7.0	10.0	10.6000	-0.6000
5	8.0	12.0	11.8667	+0.1333
6	1.0	3.0	3.0000	+0.0000
7	4.0	7.0	6.8000	+0.2000
8	6.0	8.0	9.3333	-1.3333
9	9.0	13.0	13.1333	-0.1333
10	10.0	15.0	14.4000	+0.6000

In [7]:

```

1 X_pred = 12.0
2 Y_pred = alpha + beta * X_pred
3
4 print(f"X = {X_pred} (i.e., $12,000 advertising budget)")
5 print(f"Y_hat = {alpha:.4f} + {beta:.4f} * {X_pred} = {Y_pred:.4f}
   thousand units")
6 print(f" ≈ {Y_pred * 1000:,.0f} units")
7 print()
8 print("NOTE: X = 12 is beyond the observed maximum (X_max = 10).")
9 print("This prediction is extrapolation — treat with caution.")
10 assert np.isclose(Y_pred, 16.9333333333)

```

Out[7]:

```

X = 12.0 (i.e., $12,000 advertising budget)
Y_hat = 1.7333 + 1.2667 * 12.0 = 16.9333 thousand units≈
16,933 units

```

```

NOTE: X = 12 is beyond the observed maximum (X_max = 10).
This prediction is extrapolation — treat with caution.

```

In [8]:

```

1 SS_res = np.sum(residuals ** 2)
2 SS_tot = np.sum((Y - Y_bar) ** 2)
3 R2      = 1 - SS_res / SS_tot
4
5 print(f"SS_res = {SS_res:.6f}")
6 print(f"SS_tot = {SS_tot:.4f}")
7 print(f"R2      = {R2:.6f} ({R2*100:.2f}%)")
8 print()

```



```

9 print(f"Residual mean = {residuals.mean():.6f} ≈ ( 0 → no bias)")
10 print(f"Residual std  = {residuals.std(ddof=1):.6f}")
11
12 assert np.isclose(SS_res , 3.7333333333)
13 assert np.isclose(SS_tot , 136.1)
14 assert np.isclose(R2, 0.9725691893)
15 print("\n[OK] All assertions passed")

```

Out[8]:

```

SS_res = 3.733333
SS_tot = 136.1000
R²      = 0.972569 (97.26%)

Residual mean = 0.000000 ≈ ( 0 → no bias)
Residual std  = 0.644061

[OK] All assertions passed

```

In [9]:

```

1 import numpy as np
2 import matplotlib
3
4 import matplotlib.pyplot as plt
5 import matplotlib.font_manager as fm
6 import os
7 import scipy.stats as stats
8
9 # --- 1. Robust Sarabun Font Setup ---
10 font_dir = os.path.join(os.environ.get('LOCALAPPDATA', ''), '
    Microsoft', 'Windows', 'Fonts')
11 sarabun_r_path = os.path.join(font_dir, 'Sarabun-Regular.ttf')
12 sarabun_b_path = os.path.join(font_dir, 'Sarabun-Bold.ttf')
13
14 prop_r = fm.FontProperties(fname=sarabun_r_path, size=11)
15 prop_b = fm.FontProperties(fname=sarabun_b_path, size=12)
16 prop_title = fm.FontProperties(fname=sarabun_b_path, size=14)
17 prop_small = fm.FontProperties(fname=sarabun_r_path, size=9)
18
19 def apply_font(ax):
20     for label in (ax.get_xticklabels() + ax.get_yticklabels()):
21         label.set_fontproperties(prop_r)
22
23 plt.style.use('seaborn-v0_8-whitegrid')

```

```

24
25 # --- 2. Data & OLS Calculation ---
26 X = np.array([3., 5., 2., 7., 8., 1., 4., 6., 9., 10.])
27 Y = np.array([6., 9., 4., 10., 12., 3., 7., 8., 13., 15.])
28 n = len(X)
29 months = np.arange(1, n + 1)
30
31 X_bar, Y_bar = X.mean(), Y.mean()
32 Sxx = np.sum((X - X_bar)**2)
33 Sxy = np.sum((X - X_bar)*(Y - Y_bar))
34 beta = Sxy / Sxx
35 alpha = Y_bar - beta * X_bar
36
37 Y_hat = alpha + beta * X
38 residuals = Y - Y_hat
39
40 def avoid_overlap(x, y, x_offset=0, y_offset=0):
41     return (x_offset, y_offset)
42
43 # --- Plot 1: Scatter + Regression Line ---
44 fig1, ax1 = plt.subplots(figsize=(8, 5))
45 ax1.scatter(X, Y, color='#4F46E5', s=100, edgecolors='white',
46             linewidths=1.5, label='Actual Data')
47 X_line = np.linspace(0, 13, 100)
48 ax1.plot(X_line, alpha + beta * X_line, color='#E11D48', linewidth
49         =2.5, label=fr'Regression Line: Y = {alpha:.2f} + {beta:.2f}X')
50 ax1.scatter([12], [alpha + beta * 12], color='#10B981', s=120,
51             marker='D', edgecolors='white', linewidths=1.5, label='
52             Prediction at X=12')
53
54 for i in range(n):
55     ax1.annotate(f'({X[i]:.0f}, {Y[i]:.0f})', (X[i], Y[i]),
56                 textcoords='offset points', xytext=(8, -12), fontproperties
57                 =prop_small, color='#4B5563', bbox=dict(boxstyle="round,pad
58                 =0.2", fc="white", ec="none", alpha=0.7))
59
60 ax1.set_xlabel('Advertising Budget (X) [$1,000s]', fontproperties=
61               prop_b)
62 ax1.set_ylabel('Sales (Y) [1,000 units]', fontproperties=prop_b)
63 ax1.set_title('1. Advertising Budget vs Sales', fontproperties=
64               prop_title)
65 legend = ax1.legend(prop=prop_r)
66 apply_font(ax1)

```

```

58 ax1.set_xlim(-0.5, 13.5)
59 ax1.set_ylim(0, 18)
60 plt.tight_layout()
61 fig1.savefig('plot_1_regression.pdf', dpi=300)
62 plt.close()
63
64 # --- Plot 2: Residuals Lollipop Chart ---
65 fig2, ax2 = plt.subplots(figsize=(8, 4))
66 markerline, stemlines, baseline = ax2.stem(X, residuals, linefmt='
    -', markerfmt='o', basefmt='k-')
67 plt.setp(markerline, color='#0284C7', markersize=8,
    markededgecolor='white', markededgewidth=1)
68 plt.setp(stemlines, color='#0284C7', linewidth=2)
69 plt.setp(baseline, color='#1F2937', linewidth=1.5)
70
71 for i in range(n):
72     offset = 0.15 if residuals[i] > 0 else -0.35
73     ax2.text(X[i], residuals[i] + offset, f'{residuals[i]:.2f}',
    ha='center', fontproperties=prop_small, color='#0284C7',
    bbox=dict(boxstyle="round,pad=0.1", fc="white", ec="none",
    alpha=0.8))
74
75 ax2.set_xlabel('Advertising Budget (X) [$1,000s]', fontproperties=
    prop_b)
76 ax2.set_ylabel('Residual Error ($e_i$)', fontproperties=prop_b)
77 ax2.set_title('2. Residuals (Errors) per Observation',
    fontproperties=prop_title)
78 apply_font(ax2)
79 ax2.set_xlim(-0.5, 12.5)
80 ax2.set_ylim(min(residuals) - 0.6, max(residuals) + 0.6)
81 plt.tight_layout()
82 fig2.savefig('plot_2_residuals_lollipop.pdf', dpi=300)
83 plt.close()
84
85 # --- Plot 3: Residuals vs Fitted ---
86 fig3, ax3 = plt.subplots(figsize=(6, 4.5))
87 ax3.scatter(Y_hat, residuals, color='#F59E0B', s=90, edgecolors='
    white', linewidths=1.2)
88 ax3.axhline(0, color='#1F2937', linestyle='--', linewidth=1.5)
89 for i in range(n):
90     ax3.annotate(f'M{i+1}', (Y_hat[i], residuals[i]), textcoords='
    offset points', xytext=(6, 5), fontproperties=prop_small,
    color='#4B5563')

```

```

91 ax3.set_xlabel(r'Fitted Values ( $\hat{Y}$ )', fontproperties=
    prop_b)
92 ax3.set_ylabel('Residuals', fontproperties=prop_b)
93 ax3.set_title('3. Residuals vs Fitted Values', fontproperties=
    prop_title)
94 apply_font(ax3)
95 plt.tight_layout()
96 fig3.savefig('plot_3_res_vs_fitted.pdf', dpi=300)
97 plt.close()
98
99 # --- Plot 4: Residual Distribution (Histogram) ---
100 fig4, ax4 = plt.subplots(figsize=(6, 4.5))
101 ax4.hist(residuals, bins=6, color='#8B5CF6', edgecolor='white',
    alpha=0.8, density=True)
102 x_range = np.linspace(residuals.min() - 0.5, residuals.max() +
    0.5, 100)
103 ax4.plot(x_range, stats.norm.pdf(x_range, 0, residuals.std(ddof=1)
    ), color='#E11D48', linewidth=2)
104 ax4.axvline(0, color='#1F2937', linestyle='--', linewidth=1.2)
105 ax4.set_xlabel('Residual Value', fontproperties=prop_b)
106 ax4.set_ylabel('Density', fontproperties=prop_b)
107 ax4.set_title('4. Residuals Distribution', fontproperties=
    prop_title)
108 apply_font(ax4)
109 plt.tight_layout()
110 fig4.savefig('plot_4_res_dist.pdf', dpi=300)
111 plt.close()
112
113 # --- Plot 5: Normal Q-Q Plot ---
114 fig5, ax5 = plt.subplots(figsize=(6, 4.5))
115 (osm, osr), (slope_qq, intercept_qq, _) = stats.probplot(residuals
    , dist='norm')
116 ax5.scatter(osm, osr, color='#EC4899', s=90, edgecolors='white',
    linewidths=1.2)
117 xq = np.array([osm.min(), osm.max()])
118 ax5.plot(xq, slope_qq * xq + intercept_qq, color='#1F2937',
    linewidth=2)
119 ax5.set_xlabel('Theoretical Quantiles', fontproperties=prop_b)
120 ax5.set_ylabel('Sample Quantiles', fontproperties=prop_b)
121 ax5.set_title('5. Normal Q-Q Plot', fontproperties=prop_title)
122 apply_font(ax5)
123 plt.tight_layout()
124 fig5.savefig('plot_5_qq.pdf', dpi=300)

```

```

125 plt.close()
126
127 # --- Plot 6: Scale-Location Plot ---
128 fig6, ax6 = plt.subplots(figsize=(6, 4.5))
129 sqrt_abs_res = np.sqrt(np.abs(residuals))
130 ax6.scatter(Y_hat, sqrt_abs_res, color='#10B981', s=90, edgecolors
            = 'white', linewidths=1.2)
131 for i in range(n):
132     ax6.annotate(f'M{i+1}', (Y_hat[i], sqrt_abs_res[i]),
            textcoords='offset points', xytext=(6, 5), fontproperties=
            prop_small, color='#4B5563')
133 ax6.set_xlabel(r'Fitted Values ( $\hat{Y}$ )', fontproperties=
            prop_b)
134 ax6.set_ylabel(r' $\sqrt{|\text{Residuals}|}$ ', fontproperties=prop_b)
135 ax6.set_title('6. Scale-Location Plot', fontproperties=prop_title)
136 apply_font(ax6)
137 plt.tight_layout()
138 fig6.savefig('plot_6_scale_loc.pdf', dpi=300)
139 plt.close()
140
141 # --- Plot 7: Actual vs Predicted ---
142 fig7, ax7 = plt.subplots(figsize=(6, 6))
143 sc = ax7.scatter(Y_hat, Y, c=months, cmap='plasma', s=120,
            edgecolors='white', linewidths=1.5)
144 cbar = fig7.colorbar(sc, ax=ax7)
145 cbar.set_label('Month', fontproperties=prop_b)
146 cbar.ax.tick_params(labelsize=11)
147 for t in cbar.ax.get_yticklabels():
148     t.set_fontproperties(prop_r)
149
150 lims = [min(Y.min(), Y_hat.min()) - 1, max(Y.max(), Y_hat.max()) +
            1]
151 ax7.plot(lims, lims, color='#1F2937', linestyle='--', linewidth=2,
            label='Perfect Fit')
152
153 for i in range(n):
154     ax7.annotate(f'M{i+1}', (Y_hat[i], Y[i]), textcoords='offset
            points', xytext=(-15, 8), fontproperties=prop_small, color=
            '#4B5563', bbox=dict(boxstyle="round,pad=0.2", fc="white",
            ec="none", alpha=0.6))
155
156 ax7.set_xlabel(r'Predicted Sales ( $\hat{Y}$ )', fontproperties=
            prop_b)

```

```

157 ax7.set_ylabel('Actual Sales ($Y$)', fontproperties=prop_b)
158 ax7.set_title('7. Actual vs Predicted Sales', fontproperties=
    prop_title)
159 ax7.legend(prop=prop_r)
160 apply_font(ax7)
161 ax7.set_xlim(lims)
162 ax7.set_ylim(lims)
163 ax7.set_aspect('equal')
164 plt.tight_layout()
165 fig7.savefig('plot_7_actual_vs_pred.pdf', dpi=300)
166 plt.close()
167
168 print("7 individual plots generated successfully using strict
    Sarabun FontProperties.")

```